

Yesterday 2:58 PM

Explain the four pillars of OOP with a real-world example with definition.

Abstraction vs encapsulation — what is the actual difference?

Composition vs inheritance — when would you prefer composition?

Why does Java not support multiple inheritance with classes?

How does Java achieve multiple inheritance through interfaces?

What is the difference between an abstract class and an interface?

Can an interface have constructors?

Can an abstract class have no abstract methods?

Can an abstract class be final?

Can an interface extend multiple interfaces?



Absolutely. For a **5-year Java interview**, don't just give textbook definitions. You should explain **what it means, why Java uses it, and give a practical example**.

1. Four Pillars of OOP

The four pillars are:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

Let's use a **Bank Account** example throughout.

1. Encapsulation

Definition

Encapsulation means bundling data and the methods that operate on that data inside a class, while restricting direct access to the internal state.

In Java, this is commonly achieved using:

- `private` fields
- `public/protected` methods
- controlled access through methods

Real-world example

Think about a **bank account**.

You should not be able to directly modify:

```
account.balance = -50000
```



Instead, the bank controls how the balance changes:

`</>` **Java**



```
class BankAccount {  
  
    private double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Here:

```
balance  
  ↓  
private  
  ↓  
Only BankAccount controls how it changes
```



Interview answer

"Encapsulation is about protecting an object's internal state and controlling how that state can be accessed or modified. In Java, we generally achieve it using private fields and public methods."

2. Abstraction

Definition

Abstraction means hiding implementation details and exposing only the essential behavior.

The user should know **what an object does**, not necessarily **how it does it**.

Real-world example

When you use an ATM:

You perform:

```
Insert card
↓
Enter PIN
↓
Withdraw money
```



You don't need to know:

```
How the bank validates the PIN
How the database is queried
How the transaction is processed
How the bank communicates with another service
```



That's abstraction.

Java example

```
</> Java

interface PaymentService {
```



```
void pay(double amount);  
}
```

Implementation:

```
</> Java   
  
class CreditCardPayment implements PaymentService {  
  
    @Override  
    public void pay(double amount) {  
        // validate card  
        // call payment gateway  
        // process transaction  
    }  
}
```

The caller only knows:

```
</> Java   
  
paymentService.pay(1000);
```

It doesn't care about the implementation.

Interview answer

"Abstraction focuses on exposing essential behavior while hiding implementation complexity. In Java, we commonly achieve it using interfaces and abstract classes."

3. Inheritance

Definition

Inheritance allows one class to acquire properties and behavior of another class.

It represents an **IS-A** relationship.

Example:

```
Vehicle  
↑
```



|
Car

A car is a vehicle.

</> Java



```
class Vehicle {  
  
    void start() {  
        System.out.println("Vehicle started");  
    }  
}  
  
class Car extends Vehicle {  
  
    void drive() {  
        System.out.println("Car driving");  
    }  
}
```

Now:

</> Java



```
Car car = new Car();  
  
car.start();  
car.drive();
```

Car inherits start() from Vehicle .

Real-world example

Employee
↑
|
Developer



A developer is an employee.

4. Polymorphism

Definition

Polymorphism means one interface/reference can represent multiple implementations, and the behavior can vary depending on the actual object.

"Poly" = many

"Morph" = forms

There are two major types in Java:

Compile-time polymorphism

Method overloading:

```
</> Java   
  
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Runtime polymorphism

Method overriding:

```
</> Java   
  
class Payment {  
  
    void pay() {  
        System.out.println("Generic payment");  
    }  
}  
  
class CreditCardPayment extends Payment {  
  
    @Override  
    void pay() {  
        System.out.println("Credit card payment");  
    }  
}
```

```
class UPIPayment extends Payment {  
  
    @Override  
    void pay() {  
        System.out.println("UPI payment");  
    }  
}
```

Now:

```
</> Java  
  
Payment payment;  
  
payment = new CreditCardPayment();  
payment.pay();
```

Output:

Credit card payment

And:

```
</> Java  
  
payment = new UPIPayment();  
payment.pay();
```

Output:

UPI payment

This is **runtime polymorphism**.

5. Abstraction vs Encapsulation — Actual Difference

This is a **very common** interview question.

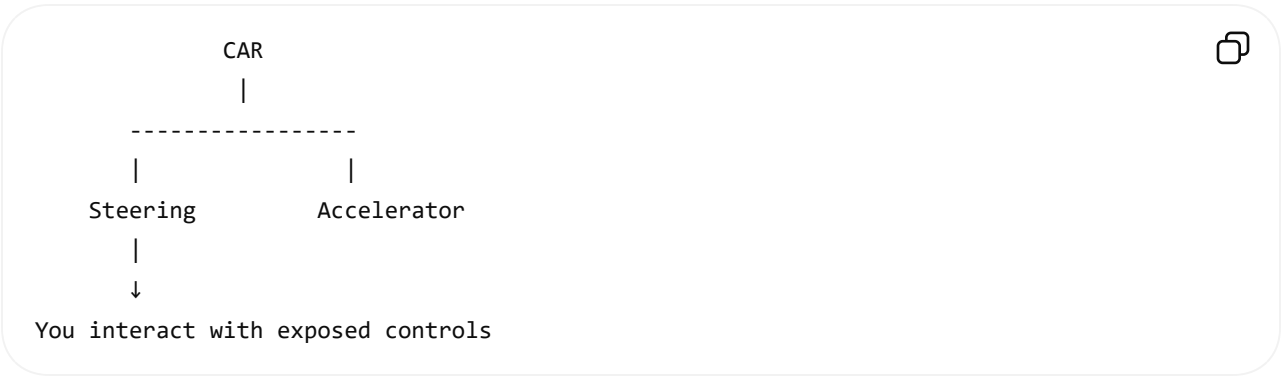
The easiest way to remember it:

Abstraction = What to expose

Encapsulation = How to protect it

Example: Car

When you drive a car:



You don't need to know how the engine works.

That's **abstraction**.

But the engine's internal state isn't directly accessible to you.

That's **encapsulation**.

Technical difference

Abstraction	Encapsulation
Hides implementation complexity	Hides/protects internal state
Focuses on what	Focuses on how data is controlled
Usually interfaces/abstract classes	Usually private fields + methods
Design-level concept	Implementation-level/data-hiding mechanism
Example: <code>PaymentService.pay()</code>	<code>private balance</code>

Strong interview answer

"Abstraction hides unnecessary implementation details and exposes only the required behavior, whereas encapsulation protects an object's internal state and controls how that state is accessed or modified."

6. Composition vs Inheritance

This is another important 5-year-level question.

Inheritance

Represents:

IS-A

Car IS-A Vehicle



</> **Java**



```
class Car extends Vehicle {  
}
```

Composition

Represents:

HAS-A

Car HAS-A Engine



</> **Java**



```
class Car {  
  
    private Engine engine;  
  
}
```

Here the Car contains an Engine .

7. When would you prefer Composition?

Generally:

Prefer composition when you want flexibility and don't have a strong IS-A relationship.

Suppose:

</> Java



```
class Car extends Engine {  
}
```

This makes no sense.

A car is **not** an engine.

Instead:

</> Java



```
class Car {  
  
    private Engine engine;  
  
}
```

Now:

Car

|

| HAS-A

↓

Engine



That's composition.

Another important example

Suppose you have:

</> Java



```
class Employee {  
}
```

And:

</> Java



```
class Developer extends Employee {  
}
```

Inheritance makes sense because:

Developer IS-A Employee



But imagine behavior like:

Employee
├─ CanFly
├─ CanDrive
└─ CanCode



Using inheritance for every combination becomes messy.

Composition allows:

`</>` **Java**



```
class Employee {  
  
    private CodingSkill codingSkill;  
    private DrivingSkill drivingSkill;  
  
}
```

You can change behavior without changing the class hierarchy.

Why composition is often preferred

Composition gives:

- Lower coupling
- Better flexibility
- Easier testing
- Easier behavior replacement
- Avoids deep inheritance hierarchies
- Better adherence to "favor composition over inheritance"

Interview answer

"I prefer composition when the relationship is HAS-A rather than IS-A, or when behavior needs to be changed independently. Composition provides better flexibility

and reduces coupling compared with deep inheritance hierarchies."

8. Why doesn't Java support Multiple Inheritance with Classes?

Java doesn't allow:

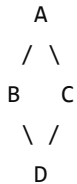
</> Java

```
class C extends A, B {  
}
```



because of the **diamond problem**.

Imagine:



Suppose:

</> Java

```
class A {  
    void show() {  
        System.out.println("A");  
    }  
}  
  
class B extends A {  
}  
  
class C extends A {  
}
```



Now if Java allowed:

</> Java



```
class D extends B, C {  
}
```

And both `B` and `C` override `show()` :

</> Java

```
B.show()  
C.show()
```



Then:

</> Java

```
D d = new D();  
d.show();
```



Which implementation should Java choose?

`B.show()`

OR

`C.show()`



This creates ambiguity.

Therefore Java deliberately doesn't support multiple inheritance of **classes**.

Interview answer

"Java doesn't support multiple inheritance through classes primarily to avoid ambiguity, especially the diamond problem, where a subclass could inherit the same method through multiple parent classes and the compiler wouldn't have a clear way to determine which implementation to use."

9. How does Java achieve Multiple Inheritance through Interfaces?

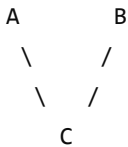
Java allows:

</> Java



```
interface A {  
    void show();  
}  
  
interface B {  
    void display();  
}  
  
class C implements A, B {  
  
    @Override  
    public void show() {  
    }  
  
    @Override  
    public void display() {  
    }  
}
```

So:



A class can implement multiple interfaces:

</> Java

```
class C implements A, B
```

What if both interfaces have the same default method?

For example:

</> Java

```
interface A {  
  
    default void show() {  
        System.out.println("A");  
    }  
}  
  
interface B {
```

```
    default void show() {  
        System.out.println("B");  
    }  
}
```

Then:

</> Java



```
class C implements A, B {  
}
```

will cause a conflict.

Java requires C to resolve it:

</> Java



```
class C implements A, B {  
  
    @Override  
    public void show() {  
        A.super.show();  
    }  
}
```

So Java can support multiple interface inheritance while providing explicit rules for resolving default-method conflicts.

10. Abstract Class vs Interface

Very important interview question.

Abstract class

</> Java



```
abstract class Vehicle {  
  
    protected String brand;  
  
    abstract void start();  
}
```

```
void stop() {  
    System.out.println("Vehicle stopped");  
}  
}
```

An abstract class can have:

- Abstract methods
- Concrete methods
- Instance variables
- Static variables
- Constructors
- Instance initialization blocks
- Static blocks
- Different access modifiers

Interface

```
</> Java   
  
interface Payment {  
  
    void pay();  
  
    default void refund() {  
        System.out.println("Refund");  
    }  
}
```

Interfaces primarily define a **contract**.

They can contain:

- Abstract methods
- default methods
- static methods
- private methods
- Constants (public static final)

But they don't have constructors because an interface itself isn't instantiated.

Key difference

Abstract Class**Interface**

Represents shared base/partial implementation

Represents contract/capability

Can have instance state

No ordinary instance fields

Can have constructors

No constructors

Can have concrete methods

Can have default/static/private methods

Class extends one abstract class

Class implements multiple interfaces

Can have protected , private members

Interface methods/fields have interface-specific rules

Good for closely related classes

Good for common capability/contract

Example

Animal

↑

Dog

Cat



An abstract `Animal` class makes sense because dogs and cats share state/behavior.

But:

Flyable

Payable

Serializable

Comparable



are capabilities/contracts, so interfaces make sense.

11. Can an Interface Have Constructors?

No.

You cannot write:

</> Java



```
interface Payment {  
  
    Payment() { // ✖  
    }  
}
```

Why?

Because interfaces aren't instantiated directly.

You can't do:

</> Java



```
new Payment(); // ✖
```

Therefore there is no object whose construction the interface needs to control.

However, an **implementing class** can have a constructor:

</> Java



```
interface Payment {  
    void pay();  
}  
  
class CreditCardPayment implements Payment {  
  
    CreditCardPayment() {  
        System.out.println("Constructor");  
    }  
  
    public void pay() {  
    }  
}
```

12. Can an Abstract Class Have No Abstract Methods?

Yes.

This is a common trick question.

</> Java



```
abstract class Vehicle {  
  
    void start() {  
        System.out.println("Starting");  
    }  
  
    void stop() {  
        System.out.println("Stopping");  
    }  
}
```

This class has **zero abstract methods**, but it is still abstract.

Why would we do this?

We may want to:

Prevent direct instantiation while providing common functionality to subclasses.

You cannot:

</> Java



```
Vehicle v = new Vehicle(); // ❌
```

But:

</> Java



```
class Car extends Vehicle {  
}  
  
Car car = new Car(); // ✅
```

This is perfectly valid.

13. Can an Abstract Class Be Final?

No.

This is a contradiction.

</> Java



```
abstract final class A {  
}
```

✗ Compilation error.

Why?

abstract means:

"This class must be extended."

final means:

"This class cannot be extended."

So:

```
abstract → MUST extend  
final    → CANNOT extend
```



They contradict each other.

Therefore Java doesn't allow:

</> Java



```
abstract final class A
```

14. Can an Interface Extend Multiple Interfaces?

Yes.

This is one of the ways Java supports multiple inheritance.

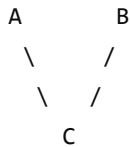
</> Java



```
interface A {  
    void methodA();  
}
```

```
interface B {  
    void methodB();  
}  
  
interface C extends A, B {  
    void methodC();  
}
```

Now:



A class implementing `C` must satisfy the contracts from all three:

</> **Java**

```
class D implements C {  
  
    public void methodA() {  
    }  
  
    public void methodB() {  
    }  
  
    public void methodC() {  
    }  
}
```

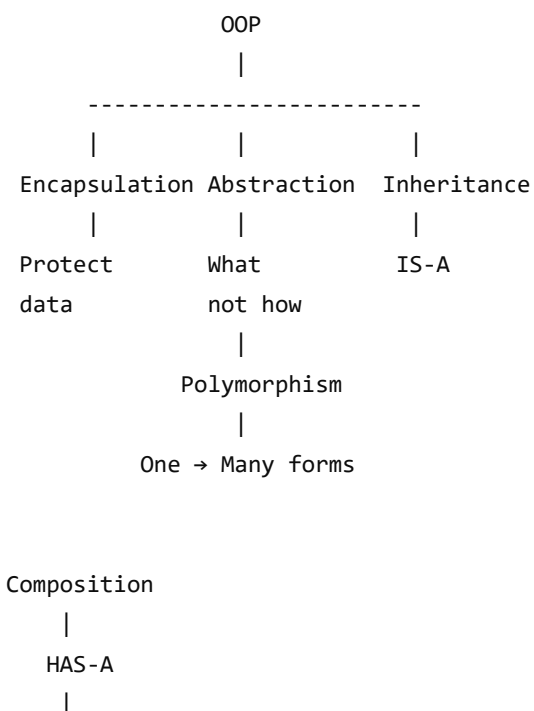
Quick Interview Revision

If the interviewer asks all of these rapidly, remember this:

Question	One-line answer
Encapsulation	Protect internal state and control access
Abstraction	Hide implementation complexity and expose essential behavior

Question	One-line answer
Inheritance	Reuse/extend behavior through an IS-A relationship
Polymorphism	Same reference/interface, different behavior
Composition	HAS-A relationship; often preferred for flexibility
Multiple class inheritance?	No, mainly to avoid ambiguity/diamond problem
Multiple interfaces?	Yes, implements A, B
Abstract class vs interface	Abstract class = shared state/implementation; interface = contract/capability
Interface constructor?	No
Abstract class with zero abstract methods?	Yes
Abstract + final?	No
Interface extends multiple interfaces?	Yes

★ Best mental model



Prefer when you want flexibility
and loose coupling

For a **5-year Java interview**, the most important follow-up questions from this set are usually **abstract class vs interface**, **composition vs inheritance**, **default-method conflicts in interfaces**, and the **diamond problem**.



Yes. These are **very important OOP + Core Java interview questions**, especially for a **4–5 year Java developer**. I'll explain them in the way you can answer in an interview: **definition** → **example** → **important rule** → **interview point**.

1. Can a class implement multiple interfaces?

Yes.

Java doesn't allow a class to extend multiple classes:

</> Java



```
class C extends A, B // ❌
```

But a class can implement multiple interfaces:

</> Java



```
interface Payment {  
    void pay();  
}
```

```
}

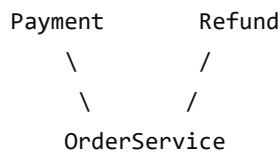
interface Refund {
    void refund();
}

class OrderService implements Payment, Refund {

    @Override
    public void pay() {
        System.out.println("Payment");
    }

    @Override
    public void refund() {
        System.out.println("Refund");
    }
}
```

So:



Why is this allowed?

Because interfaces primarily define contracts, and Java provides rules for resolving conflicts when interfaces have implementations through `default` methods.

Interview answer

"Yes, a Java class can implement multiple interfaces. This allows a class to satisfy multiple contracts without the ambiguity associated with multiple class inheritance."

2. What happens when two interfaces contain the same default method?

This is a **very important** interview question.

Suppose:

</> Java



```
interface A {  
  
    default void show() {  
        System.out.println("A");  
    }  
}  
  
interface B {  
  
    default void show() {  
        System.out.println("B");  
    }  
}
```

Now:

</> Java



```
class C implements A, B {  
  
}
```

Java doesn't know whether `c.show()` should use `A` or `B`.

Therefore:

Compilation error

You must override the method in `C`.

</> Java



```
class C implements A, B {  
  
    @Override  
    public void show() {  
        System.out.println("C");  
    }  
}
```

Now the ambiguity is resolved.

Can I explicitly call A's implementation?

Yes.

</> **Java**



```
class C implements A, B {  
  
    @Override  
    public void show() {  
        A.super.show();  
    }  
}
```

This means:

C.show()



A.show()



Similarly:

</> **Java**



```
B.super.show();
```

would invoke B 's default implementation.

Interview answer

"If two interfaces provide the same default method, the implementing class must override that method to resolve the ambiguity. Inside the overriding method, it can explicitly call `InterfaceName.super.method()` ."

3. What is Method Overloading?

Definition

Method overloading means having multiple methods with the same name but different parameter lists in the same class.

Example:

</> Java



```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

Same method name:

add()



Different parameter lists:

```
add(int, int)  
add(int, int, int)  
add(double, double)
```



Java determines which method to call based on the arguments.

What can differ?

You can overload based on:

- Number of parameters
- Parameter types
- Order of parameter types

Example:

</> Java



```
void process(int id, String name)  
  
void process(String name, int id)
```

These are different methods.

Can we overload only by changing return type?

No.

This is invalid:

```
</> Java   
  
int getValue() {  
    return 10;  
}  
  
String getValue() {  
    return "10";  
}
```

The compiler cannot distinguish methods using only return type.

Interview answer

"Method overloading is compile-time polymorphism where multiple methods have the same name but different parameter lists. Return type alone cannot be used for overloading."

4. What is Method Overriding?

Definition

Method overriding occurs when a subclass provides its own implementation of a method inherited from its parent class.

Example:

```
</> Java   
  
class Payment {  
  
    void pay() {  
        System.out.println("Generic payment");  
    }  
}
```

```
}  
}
```

Child:

```
</> Java  
  
class UPIPayment extends Payment {  
  
    @Override  
    void pay() {  
        System.out.println("UPI payment");  
    }  
}
```

Now:

```
</> Java  
  
Payment payment = new UPIPayment();  
  
payment.pay();
```

Output:

```
UPI payment
```

The child implementation is executed.

5. Overloading vs Overriding

This is one of the **most frequently asked Java questions**.

Feature	Overloading	Overriding
Meaning	Same method name, different parameters	Child replaces parent implementation
Polymorphism	Compile-time	Runtime
Inheritance required?	No	Yes
Parameter list	Must be different	Must be same

Feature	Overloading	Overriding
Return type	Can differ, subject to valid signature rules	Same or covariant
Static methods	Can be overloaded	Cannot truly be overridden
Private methods	Can be overloaded	Cannot be overridden
Final methods	Can be overloaded	Cannot be overridden
Binding	Compile time	Runtime

Simple example

Overloading:

`</>` Java



```
calculate(int a, int b)
calculate(double a, double b)
```

Compiler decides.

Overriding:

`</>` Java



```
Payment p = new UPIPayment();

p.pay();
```

JVM chooses the implementation based on the **actual object**.

6. Can you override a static method?

No.

Static methods belong to the **class**, not the object.

Example:

</> Java



```
class Parent {  
  
    static void show() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
  
    static void show() {  
        System.out.println("Child");  
    }  
}
```

This is **method hiding**, not overriding.

Consider:

</> Java



```
Parent p = new Child();  
  
p.show();
```

Output:

Parent



Why?

Because static method resolution is based on the **reference type**, not the actual object.

Reference type → Parent



Parent.show()



Interview answer

"Static methods cannot be overridden because they belong to the class rather than an object. A subclass can define a static method with the same signature, but that is method hiding, not runtime overriding."

7. Can you override a private method?

No.

Private methods are not inherited by subclasses.

</> Java



```
class Parent {  
  
    private void show() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
  
    private void show() {  
        System.out.println("Child");  
    }  
}
```

The `show()` methods are completely independent.

It is **not overriding**.

Think:

Parent

└─ private show()

Child

└─ private show()

They are unrelated methods.



Interview answer

"No. A private method is not visible to or inherited by the subclass, so it cannot be overridden."

8. Can you override a final method?

No.

If the parent says:

```
</> Java   
  
class Parent {  
  
    final void show() {  
        System.out.println("Parent");  
    }  
}
```

Child cannot do:

```
</> Java   
  
class Child extends Parent {  
  
    void show() { //   
        System.out.println("Child");  
    }  
}
```

Compiler error.

Why?

final means:

"This implementation cannot be changed by subclasses."

Interview answer

"No. A final method cannot be overridden because final explicitly prevents subclasses from changing its implementation."

9. Can a covariant return type be used while overriding?

Yes.

This is an important Java feature.

Suppose:

</> Java



```
class Animal {  
}
```

And:

</> Java



```
class Dog extends Animal {  
}
```

Parent:

</> Java



```
class Parent {  
  
    Animal getAnimal() {  
        return new Animal();  
    }  
}
```

Child:

</> Java



```
class Child extends Parent {  
  
    @Override  
    Dog getAnimal() {  
        return new Dog();  
    }  
}
```

This is valid.

Why?

Because:

Dog IS-A Animal



So the child return type can be a **subtype** of the parent's return type.

That's called a **covariant return type**.

What isn't allowed?

Parent:

</> Java

```
Animal getAnimal()
```



Child:

</> Java

```
String getAnimal() // ❌
```



because `String` isn't a subtype of `Animal`.

Interview answer

"Yes. During method overriding, the child can return a subtype of the parent's return type. This is called a covariant return type."

10. What is Runtime Polymorphism?

Definition

Runtime polymorphism means:

The method implementation that gets executed is determined at runtime based on the actual object rather than the reference type.

Example:

</> Java

```
class Payment {  
  
    void pay() {  
        System.out.println("Payment");  
    }  
}
```



```
}

class CardPayment extends Payment {

    @Override
    void pay() {
        System.out.println("Card payment");
    }
}

class UPIPayment extends Payment {

    @Override
    void pay() {
        System.out.println("UPI payment");
    }
}
```

Now:

 **Java**



```
Payment payment;

payment = new CardPayment();
payment.pay();
```

Output:

Card payment



Then:

 **Java**



```
payment = new UPIPayment();
payment.pay();
```

Output:

UPI payment



Same reference:

Payment



Different actual objects:

CardPayment
UPIPayment



Different behavior.

That's runtime polymorphism.

11. Explain Dynamic Method Dispatch

Dynamic method dispatch is essentially the mechanism behind **runtime method overriding**.

Suppose:

`</> Java`



```
Payment payment = new UPIPayment();
```

There are two types involved:

Reference type
↓
Payment

Actual object
↓
UPIPayment



When you call:

`</> Java`



```
payment.pay();
```

Java doesn't simply say:

```
Reference = Payment  
therefore execute Payment.pay()
```



Instead, for an overridden instance method, the JVM dispatches the call to the implementation corresponding to the actual object:

```

payment.pay()
  ↓
actual object = UPIPayment
  ↓
UPIPayment.pay()

```



That's **dynamic method dispatch**.

Very important interview distinction

</> **Java**



```
Payment payment = new UPIPayment();
```

Compile time:

The compiler checks whether `Payment` has a `pay()` method.

Runtime:

The JVM determines which overridden implementation to execute based on the actual object.

```

Compile time
  ↓
Is pay() available in Payment?
  ↓
  YES

Runtime
  ↓
What object is actually there?
  ↓
  UPIPayment
  ↓
UPIPayment.pay()

```



12. Tight Coupling vs Loose Coupling

This becomes extremely important when discussing **Spring Boot** and **microservices**.

Tight Coupling

Two classes are tightly coupled when one class directly depends heavily on a specific implementation of another class.

Example:

</> Java



```
class OrderService {  
  
    private MySQLOrderRepository repository =  
        new MySQLOrderRepository();  
  
}
```

OrderService is directly tied to:

MySQLOrderRepository



If you want to change to:

MongoOrderRepository



you need to modify OrderService .

That's tight coupling.

Loose Coupling

Instead, depend on an abstraction:

</> Java



```
interface OrderRepository {  
  
    void save(Order order);  
  
}
```

Implementation:

</> Java

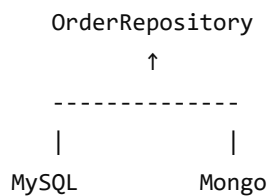


```
class MySQLOrderRepository implements OrderRepository {  
  
    public void save(Order order) {  
        // MySQL Logic  
    }  
}
```

Service:

```
</> Java  
  
class OrderService {  
  
    private final OrderRepository repository;  
  
    OrderService(OrderRepository repository) {  
        this.repository = repository;  
    }  
}
```

Now:



OrderService doesn't care which implementation is used.

This is loose coupling.

13. What is Dependency Inversion?

This is an important **SOLID** principle.

Definition

Dependency Inversion Principle (DIP) says:

High-level modules should not depend directly on low-level modules. Both should depend on abstractions.

And:

Abstractions should not depend on details; details should depend on abstractions.

Without Dependency Inversion

</> Java



```
class OrderService {  
  
    private MySQLRepository repository =  
        new MySQLRepository();  
  
}
```

Dependency:

```
OrderService  
    ↓  
MySQLRepository
```



The high-level business logic depends directly on a low-level implementation.

With Dependency Inversion

Create abstraction:

</> Java



```
interface OrderRepository {  
  
    void save(Order order);  
  
}
```

Implementation:

</> Java



```
class MySQLRepository implements OrderRepository {  
  
    public void save(Order order) {  
        // save to MySQL  
    }  
  
}
```

```
}
}
```

Service:

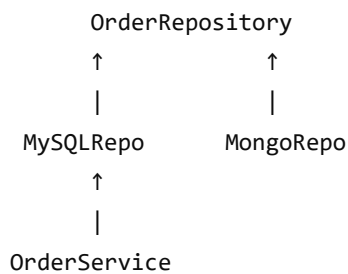
```
</> Java

class OrderService {

    private final OrderRepository repository;

    OrderService(OrderRepository repository) {
        this.repository = repository;
    }
}
```

Now:



The service depends on:

OrderRepository

not:

MySQLRepository

14. Dependency Inversion vs Dependency Injection

This is a very common follow-up.

Don't confuse them.

Dependency Inversion

A design principle.

It says:

Depend on abstraction,
not concrete implementation.



Dependency Injection

A **technique/mechanism** for providing dependencies from outside.

For example, Spring:

</> **Java**



```
@Service
class OrderService {

    private final OrderRepository repository;

    @Autowired
    OrderService(OrderRepository repository) {
        this.repository = repository;
    }
}
```

Spring creates the dependency and injects it.

So:

Dependency Inversion
↓
Design principle
↓
Depend on abstraction

Dependency Injection
↓
Implementation technique
↓
Provide dependency from outside



Excellent 5-year interview answer

"Dependency Inversion is a SOLID design principle where high-level modules depend on abstractions rather than concrete implementations. Dependency Injection is one

technique used to achieve that principle by supplying dependencies externally, which is why frameworks like Spring make DI very convenient."

Final Interview Cheat Sheet

Multiple Interfaces

↓

class C implements A, B

Same default method

↓

Child must resolve conflict

Overloading

↓

Same name + different parameters

↓

Compile time

Overriding

↓

Child provides implementation

↓

Runtime

static

↓

Cannot override

↓

Method hiding

private

↓

Cannot override

↓

Not inherited

final

↓

Cannot override

Covariant return

↓

Child return type can be subtype

↓



Dog instead of Animal

Runtime Polymorphism

↓

Parent ref → Child object

↓

Child implementation executes

Dynamic Dispatch

↓

JVM selects overridden method

↓

based on actual object

Tight Coupling

↓

Depend directly on implementation

Loose Coupling

↓

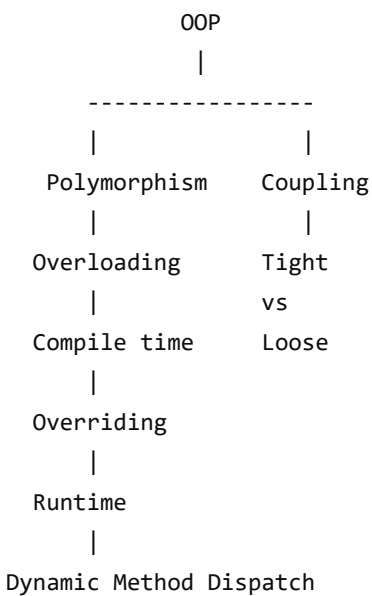
Depend on abstraction

Dependency Inversion

↓

High-level → Abstraction ← Low-level

One particularly important distinction to remember



For a **5-year Java interview**, after these questions, interviewers commonly move into `equals()` / `hashCode()` , **immutable classes**, **String internals**, `final` / `finally` / `finalize` ,

exception handling, Java 8 functional interfaces, streams, and multithreading.



Explain SOLID principles with Java examples.

Explain Open/Closed Principle with a practical example.

How would you refactor a class violating SRP?

What is the difference between IS-A and HAS-A relationships?

Scenario questions

You have a PaymentService supporting UPI, Card and Net Banking. How would you design it so adding a new payment type doesn't require modifying existing code?

You have a class with 20 different if-else conditions based on paymentType. How would you refactor it?

For a 5-year Java interview, SOLID is not just about memorizing five definitions. The interviewer usually wants to see whether you can **identify bad design and refactor it**.

I'll use a **payment system** because it maps directly to your scenarios.

1. What are SOLID Principles?

SOLID is a set of five object-oriented design principles that help us build code that is:

- Maintainable
- Extensible
- Testable
- Loosely coupled
- Easier to change

S → Single Responsibility Principle

O → Open/Closed Principle

L → Liskov Substitution Principle

I → Interface Segregation Principle

D → Dependency Inversion Principle



2. S — Single Responsibility Principle

Definition

A class should have one responsibility and one reason to change.

It doesn't necessarily mean a class can have only one method.

It means the class should have **one cohesive responsibility**.

Bad example

Suppose we have:

</> Java



```
class PaymentService {  
  
    public void processPayment() {  
        // payment logic  
    }  
  
    public void savePaymentToDatabase() {  
        // database logic  
    }  
  
    public void sendEmail() {  
        // email logic  
    }  
  
    public void generateInvoice() {  
        // invoice logic  
    }  
}
```

This class has many responsibilities:

PaymentService



- |
- |— Payment processing
- |— Database
- |— Email
- |— Invoice

Therefore, it has multiple reasons to change.

Refactored

Separate responsibilities:

</> **Java** 

```
class PaymentService {  
  
    public void processPayment() {  
        // payment logic  
    }  
}
```

</> **Java** 

```
class PaymentRepository {  
  
    public void savePayment() {  
        // database logic  
    }  
}
```

</> **Java** 

```
class EmailService {  
  
    public void sendEmail() {  
        // email logic  
    }  
}
```

</> **Java** 

```
class InvoiceService {  
  
    public void generateInvoice() {  
        // invoice logic  
    }  
}
```

Now:

PaymentService

|
| payment responsibility

PaymentRepository

|
| persistence responsibility

EmailService

|
| email responsibility

InvoiceService

|
| invoice responsibility



Interview answer

"SRP means a class should have one cohesive responsibility and therefore one primary reason to change. If a class handles payment, persistence, notification and invoice generation, I would split those responsibilities into separate classes."

3. O — Open/Closed Principle

Definition

Software entities should be open for extension but closed for modification.

Meaning:

New requirement

↓

Extend existing behavior

↓

Don't keep modifying stable existing code



4. Practical Payment Example

Suppose initially we support:

UPI
Card
Net Banking



Bad implementation:

</> Java



```
class PaymentService {  
  
    public void pay(String paymentType) {  
  
        if (paymentType.equals("UPI")) {  
            // UPI Logic  
        }  
        else if (paymentType.equals("CARD")) {  
            // Card Logic  
        }  
        else if (paymentType.equals("NET_BANKING")) {  
            // Net banking Logic  
        }  
    }  
}
```

Now suppose tomorrow we add:

Wallet
Crypto
PayLater



We have to keep modifying this class.

That's a violation of OCP.

5. Apply OCP

Create an abstraction:

</> Java



```
interface PaymentProcessor {
```

```
void pay(double amount);  
}
```

UPI:

```
</> Java   
  
class UpiPaymentProcessor implements PaymentProcessor {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("Processing UPI payment");  
    }  
}
```

Card:

```
</> Java   
  
class CardPaymentProcessor implements PaymentProcessor {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("Processing card payment");  
    }  
}
```

Net Banking:

```
</> Java   
  
class NetBankingPaymentProcessor implements PaymentProcessor {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("Processing net banking payment");  
    }  
}
```

Service:

```
</> Java   
  
class PaymentService {  
  
    private final PaymentProcessor processor;
```

```
public PaymentService(PaymentProcessor processor) {  
    this.processor = processor;  
}  
  
public void pay(double amount) {  
    processor.pay(amount);  
}  
}
```

Now adding Wallet:

</> Java



```
class WalletPaymentProcessor implements PaymentProcessor {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("Processing wallet payment");  
    }  
}
```

We don't modify:

UPI processor
Card processor
Net Banking processor
PaymentService



We simply add a new implementation.

That's the **Open/Closed Principle**.

6. L — Liskov Substitution Principle

Definition

Objects of a subclass should be replaceable for objects of the parent type without breaking the correctness of the program.

In simple words:

If B is a subtype of A, wherever A is expected, B should work correctly.

Classic bad example

```
</> Java   
  
class Bird {  
  
    void fly() {  
        System.out.println("Flying");  
    }  
}
```

Then:

```
</> Java   
  
class Sparrow extends Bird {  
}
```

Fine.

But:

```
</> Java   
  
class Penguin extends Bird {  
  
    @Override  
    void fly() {  
        throw new UnsupportedOperationException();  
    }  
}
```

Now:

```
</> Java   
  
Bird bird = new Penguin();  
  
bird.fly();
```

The program breaks.

The abstraction `Bird` promised that its birds can fly, but Penguin can't.

That's a violation of LSP.

Better design

</> Java



```
class Bird {  
}
```

Separate flying capability:

</> Java



```
interface Flyable {  
  
    void fly();  
}
```

Then:

</> Java



```
class Sparrow extends Bird implements Flyable {  
  
    @Override  
    public void fly() {  
        System.out.println("Flying");  
    }  
}
```

Penguin:

</> Java



```
class Penguin extends Bird {  
}
```

Now the hierarchy doesn't make a false promise.

Interview answer

"LSP means a subtype should be safely substitutable for its parent without changing expected behavior. If a subclass requires callers to add special checks or throws

unsupported-operation exceptions for behavior promised by the parent, the abstraction may violate LSP."

7.1 — Interface Segregation Principle

Definition

Clients should not be forced to depend on methods they don't need.

Bad:

```
</> Java

interface Employee {

    void code();

    void test();

    void manage();

    void deploy();

}
```



Now a tester might only need:

```
test()
```



but is forced to implement:

```
code()
manage()
deploy()
```



Better

Split interfaces:

```
</> Java
```



```
interface Developer {  
  
    void code();  
}
```

</> Java



```
interface Tester {  
  
    void test();  
}
```

</> Java



```
interface Manager {  
  
    void manage();  
}
```

Now classes implement only what they actually need.

Interview answer

"ISP says interfaces should be small and focused so that clients don't have to implement methods they don't use."

8. D — Dependency Inversion Principle

Definition

High-level modules should not depend directly on low-level implementations. Both should depend on abstractions.

Bad:

</> Java



```
class PaymentService {  
  
    private StripePaymentGateway gateway =  
        new StripePaymentGateway();  
}
```

Now:

```
PaymentService
  ↓
StripePaymentGateway
```



PaymentService is tightly coupled to Stripe.

Better

Create abstraction:

```
</> Java

interface PaymentGateway {

    void processPayment(double amount);
}
```



Implementation:

```
</> Java

class StripePaymentGateway implements PaymentGateway {

    @Override
    public void processPayment(double amount) {
        // Stripe Logic
    }
}
```



Service:

```
</> Java

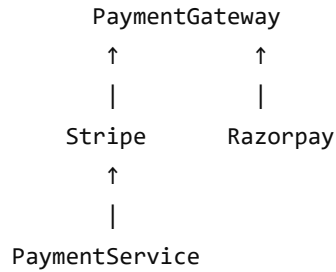
class PaymentService {

    private final PaymentGateway gateway;

    public PaymentService(PaymentGateway gateway) {
        this.gateway = gateway;
    }
}
```



Now:



The high-level service depends on the abstraction.

9. SRP vs OCP vs DIP — Don't Confuse Them

A good way to distinguish them:

SRP

↓

Separate responsibilities

OCP

↓

Add new behavior without modifying stable code

LSP

↓

Subtypes must behave correctly as their parent type

ISP

↓

Don't force clients to depend on unnecessary methods

DIP

↓

Depend on abstractions instead of concrete implementations

10. IS-A vs HAS-A

This is extremely important when discussing **inheritance vs composition**.

IS-A

Represents inheritance.

Dog IS-A Animal



Java:

</> **Java**



```
class Animal {  
}  
  
class Dog extends Animal {  
}
```

Here:

</> **Java**



```
Dog dog = new Dog();  
Animal animal = dog;
```

This is valid because Dog is an Animal.

HAS-A

Represents composition.

Car HAS-A Engine



Java:

</> **Java**



```
class Engine {  
}  
  
class Car {  
    private Engine engine;  
}
```

Here:

```

Car
|
| HAS-A
↓
Engine

```



A car isn't an engine.

It has an engine.

Interview shortcut

Ask yourself:

"Is the child actually a specialized version of the parent?"

If yes:

```
IS-A → inheritance
```



If the object contains another object:

```
HAS-A → composition
```



11. Scenario: PaymentService with UPI, Card and Net Banking

This is probably the **most important question in your list.**

Interview question

"You have a PaymentService supporting UPI, Card and Net Banking. How would you design it so adding a new payment type doesn't require modifying existing code?"

Don't do this

```

</> Java

if (type.equals("UPI")) {
}

```



```
else if (type.equals("CARD")) {  
}  
else if (type.equals("NET_BANKING")) {  
}
```

Because every new payment type requires modifying the existing service.

12. Strategy Pattern + OCP

I'd use the **Strategy Pattern**.

Create:

```
</> Java   
  
interface PaymentStrategy {  
  
    void pay(double amount);  
}
```

UPI:

```
</> Java   
  
class UpiPayment implements PaymentStrategy {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("UPI payment: " + amount);  
    }  
}
```

Card:

```
</> Java   
  
class CardPayment implements PaymentStrategy {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("Card payment: " + amount);  
    }  
}
```

Net Banking:

</> **Java**



```
class NetBankingPayment implements PaymentStrategy {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("Net banking payment: " + amount);  
    }  
}
```

Service:

</> **Java**



```
class PaymentService {  
  
    private final PaymentStrategy paymentStrategy;  
  
    public PaymentService(PaymentStrategy paymentStrategy) {  
        this.paymentStrategy = paymentStrategy;  
    }  
  
    public void processPayment(double amount) {  
        paymentStrategy.pay(amount);  
    }  
}
```

Usage:

</> **Java**



```
PaymentStrategy strategy =  
    new UpiPayment();  
  
PaymentService service =  
    new PaymentService(strategy);  
  
service.processPayment(1000);
```

Tomorrow:

</> **Java**



```
class WalletPayment implements PaymentStrategy {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("Wallet payment");  
    }  
}
```

No modification required in:

PaymentService
UpiPayment
CardPayment
NetBankingPayment



That's a clean application of:

OCP + Strategy Pattern + DIP



13. Scenario: 20 if-else Conditions Based on paymentType

Suppose you have this monster:

</> Java



```
class PaymentService {  
  
    public void process(String paymentType, double amount) {  
  
        if (paymentType.equals("UPI")) {  
            // 20 lines  
        }  
        else if (paymentType.equals("CARD")) {  
            // 30 lines  
        }  
        else if (paymentType.equals("NET_BANKING")) {  
            // 25 lines  
        }  
        // ...  
        // 20 conditions  
    }  
}
```

This is a **code smell**.

The first thing I'd think about is:

Strategy Pattern



14. Refactor Using Strategy + Factory/Registry

Create:

</> Java



```
interface PaymentStrategy {  
  
    void process(double amount);  
}
```

Implement each payment type separately:

</> Java



```
class UpiPaymentStrategy implements PaymentStrategy {  
  
    @Override  
    public void process(double amount) {  
        // UPI-specific logic  
    }  
}
```

</> Java



```
class CardPaymentStrategy implements PaymentStrategy {  
  
    @Override  
    public void process(double amount) {  
        // Card-specific logic  
    }  
}
```

</> Java



```
class NetBankingPaymentStrategy implements PaymentStrategy {  
  
    @Override
```

```
public void process(double amount) {  
    // Net banking-specific logic  
}  
}
```

Now we need to select the correct strategy.

A simple approach:

</> Java



```
class PaymentStrategyFactory {  
  
    private final Map<String, PaymentStrategy> strategies;  
  
    public PaymentStrategyFactory(  
        Map<String, PaymentStrategy> strategies) {  
        this.strategies = strategies;  
    }  
  
    public PaymentStrategy getStrategy(String type) {  
  
        PaymentStrategy strategy = strategies.get(type);  
  
        if (strategy == null) {  
            throw new IllegalArgumentException(  
                "Unsupported payment type: " + type);  
        }  
  
        return strategy;  
    }  
}
```

Then:

</> Java



```
class PaymentService {  
  
    private final PaymentStrategyFactory factory;  
  
    public PaymentService(PaymentStrategyFactory factory) {  
        this.factory = factory;  
    }  
  
    public void process(String type, double amount) {
```

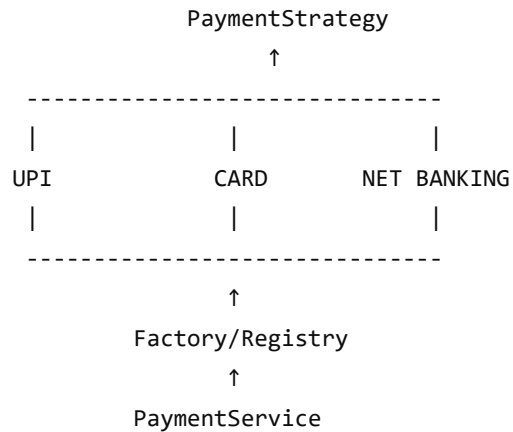
```

        PaymentStrategy strategy =
            factory.getStrategy(type);

        strategy.process(amount);
    }
}

```

Now the architecture becomes:



15. In Spring Boot, I'd Go One Step Further

In a real Spring Boot application, I wouldn't manually instantiate these strategies.

I could use Spring's dependency injection and a registry/map.

For example:

</> Java

```

interface PaymentStrategy {

    String getType();

    void process(double amount);
}

```

UPI:

</> Java

```

@Component
class UpiPaymentStrategy implements PaymentStrategy {

```

```
@Override
public String getType() {
    return "UPI";
}

@Override
public void process(double amount) {
    // UPI Logic
}
}
```

Card:

```
</> Java 
@Component
class CardPaymentStrategy implements PaymentStrategy {

    @Override
    public String getType() {
        return "CARD";
    }

    @Override
    public void process(double amount) {
        // Card Logic
    }
}
```

Service:

```
</> Java 
@Service
class PaymentService {

    private final Map<String, PaymentStrategy> strategies;

    public PaymentService(
        List<PaymentStrategy> strategyList) {

        this.strategies = strategyList.stream()
            .collect(Collectors.toMap(
                PaymentStrategy::getType,
                Function.identity()
            ));
    }
}
```

```

    }

    public void process(String type, double amount) {

        PaymentStrategy strategy = strategies.get(type);

        if (strategy == null) {
            throw new IllegalArgumentException(
                "Unsupported payment type: " + type);
        }

        strategy.process(amount);
    }
}

```

Now if tomorrow you add:

```

</> Java

@Component
class WalletPaymentStrategy implements PaymentStrategy {

    @Override
    public String getType() {
        return "WALLET";
    }

    @Override
    public void process(double amount) {
        // Wallet Logic
    }
}

```

You don't modify `PaymentService`.

Spring discovers the new implementation and injects it into the list.



How I'd Answer This Scenario in an Interview

If the interviewer asks:

"How would you refactor a class with 20 if-else conditions based on paymentType?"

A strong 5-year answer would be:

"I would first identify whether each condition represents a different business behavior. If it does, I would use the Strategy Pattern. I'd define a `PaymentStrategy` interface and create separate implementations for UPI, Card, Net Banking, etc. Then I'd use a Factory or, in Spring Boot, a map-based strategy registry to select the implementation. This removes the large conditional block, follows Open/Closed and Dependency Inversion principles, and allows new payment types to be added without modifying the existing payment service."

That's much stronger than simply saying:

"I'll replace if-else with switch."

★ One Important Interview Insight

If you see:

</> **Java**



```
if (type.equals("A")) {  
    behavior A  
}  
else if (type.equals("B")) {  
    behavior B  
}  
else if (type.equals("C")) {  
    behavior C  
}
```

Don't automatically say **Factory**.

Ask:

"Are these different behaviors?"

If yes → **Strategy Pattern**

If you're mainly deciding **which object to create** → **Factory Pattern**

If objects are selected and registered dynamically → **Factory + Strategy/Registry** can be appropriate.

For your payment scenario:

Payment Type



Select behavior



Strategy Pattern



Factory/Registry can select strategy



That's the kind of distinction that usually separates a **basic Java answer** from a **5-year experienced answer**.



When would you choose:
interface
abstract class
composition
inheritance?

For a **5-year Java interview**, the easiest way to decide is to ask **what relationship/design problem you have**.

Quick decision rule

Do I need a CONTRACT / CAPABILITY?



Interface

Do I need a COMMON BASE with shared state/behavior?



Abstract class

Is this object made up of / HAS-A another object?



Composition

Is this genuinely an IS-A specialization?



Inheritance



1. When would you choose an interface ?

Choose an **interface** when you want to define a **contract/capability** that potentially unrelated classes can implement.

Example: Payment

</> Java



```
interface PaymentProcessor {  
    void pay(double amount);  
}
```

Different implementations:

</> Java



```
class UpiPayment implements PaymentProcessor {  
    public void pay(double amount) {  
        // UPI Logic  
    }  
}  
  
class CardPayment implements PaymentProcessor {  
    public void pay(double amount) {  
        // Card Logic  
    }  
}
```

Here the important thing is:

"Any payment processor must provide `pay()` ."

Good use cases

PaymentProcessor
NotificationSender
MessagePublisher
Cache
Repository
Serializable
Comparable
Runnable



Choose interface when:

- You need a **contract**
- Multiple unrelated classes need the same capability
- You want loose coupling
- You want multiple inheritance of type
- You want implementations to be easily replaceable

Interview answer

"I choose an interface when I want to define a contract or capability and don't want the consumer to depend on a particular implementation."

2. When would you choose an abstract class ?

Choose an **abstract class** when multiple classes have a **strong conceptual relationship** and need to share **state and common implementation**.

Example:

</> Java



```
abstract class Vehicle {  
  
    protected String registrationNumber;  
  
    public Vehicle(String registrationNumber) {  
        this.registrationNumber = registrationNumber;  
    }  
  
    public void stop() {  
        System.out.println("Vehicle stopped");  
    }  
  
    abstract void start();  
}
```

Then:

</> Java



```
class Car extends Vehicle {  
  
    public Car(String registrationNumber) {  
        super(registrationNumber);  
    }  
}
```

```
}

@Override
void start() {
    System.out.println("Car started");
}
}
```

Here Car and Vehicle have a genuine:

Car IS-A Vehicle



relationship.

Abstract class is useful when you need:

- Shared instance fields
- Constructors
- Common implementation
- Protected helper methods
- Some abstract methods + some concrete methods
- A controlled base-class hierarchy

Interview answer

"I choose an abstract class when subclasses share common state or implementation and have a strong IS-A relationship. It allows me to provide partial implementation while forcing subclasses to implement specific behavior."

3. When would you choose Composition?

Choose **composition** when one object **HAS-A** another object.

Example:

```
</> Java

class Car {

    private Engine engine;

    public Car(Engine engine) {
        this.engine = engine;
    }
}
```



```

    }

    public void start() {
        engine.start();
    }
}

```

Relationship:

```

Car
|
| HAS-A
↓
Engine

```



A car isn't an engine.

Therefore:

</> Java

```
class Car extends Engine // ❌ Bad design
```



would be incorrect.

Composition is usually preferable when you want to **assemble behavior from independent components**.

Real-world example

```

OrderService
|
├─ PaymentService
├─ InventoryService
├─ NotificationService
└─ OrderRepository

```



You could have:

</> Java

```
class OrderService {

    private final PaymentService paymentService;
    private final InventoryService inventoryService;

```



```
private final NotificationService notificationService;  
}
```

That's composition.

Choose composition when:

- Relationship is HAS-A
- You want loose coupling
- Components can be replaced independently
- You want easier testing
- Behavior may change dynamically
- You want to avoid deep inheritance hierarchies

Interview answer

"I generally prefer composition when an object needs another object's functionality. It gives better flexibility and lower coupling because I can replace the composed dependency without changing the class hierarchy."

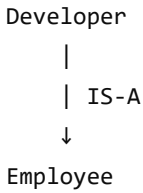
4. When would you choose Inheritance?

Choose inheritance when there is a **genuine IS-A relationship** and the child is a true specialization of the parent.

Example:

```
</> Java   
  
class Employee {  
  
    void work() {  
        System.out.println("Working");  
    }  
}  
  
class Developer extends Employee {  
  
    void writeCode() {  
        System.out.println("Writing code");  
    }  
}
```

Relationship:



You can safely do:

</> Java

```
Employee employee = new Developer();
```

because a Developer really is an Employee .

Choose inheritance when:

- There is a strong IS-A relationship
- Subclass genuinely satisfies the parent contract
- Shared behavior/state is meaningful
- Polymorphism is useful
- The hierarchy is stable and not artificially created for code reuse

Don't use inheritance just for code reuse.

This is a common interview point.

Suppose:

</> Java

```
class Car extends Engine
```

just because you want to reuse:

</> Java

```
engine.start();
```

That's bad design.

Use:

</> Java

```
class Car {  
    private Engine engine;  
}
```

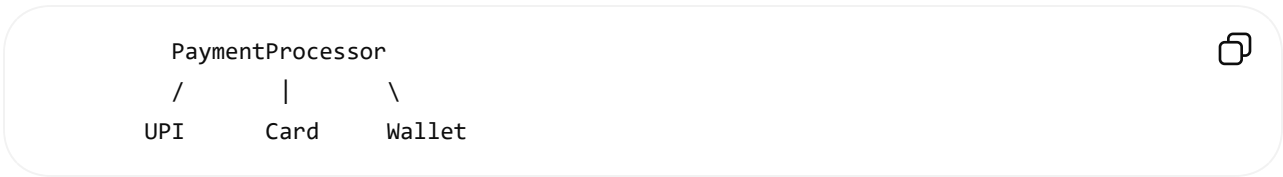
instead.

🔥 Interface vs Abstract Class

This is where interviewers often dig deeper.

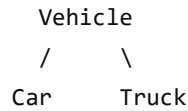
	Interface	Abstract Class
Main purpose	Contract/capability	Common base + partial implementation
State	No ordinary instance state	Can have instance state
Constructor	❌ No	✅ Yes
Multiple inheritance	A class can implement multiple interfaces	A class can extend only one class
Concrete methods	default , static , private methods	Yes
Fields	Constants	Instance/static fields
Best for	Loose coupling	Closely related classes

Example



I'd likely use an **interface** because payment types are implementations of a contract.

But:

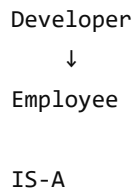


If they share substantial state and implementation, an **abstract class** may make sense.

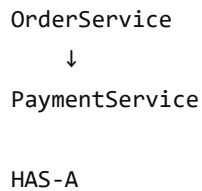
Composition vs Inheritance

This is another major interview topic.

Inheritance



Composition



The classic rule:

Favor composition over inheritance when inheritance doesn't represent a strong, substitutable IS-A relationship.

One Practical Example

Suppose you're designing an e-commerce system.

Payment types

</> Java



```
interface PaymentProcessor {  
    void pay(double amount);  
}
```

✓ Interface

Because:

UPI
Card
NetBanking
Wallet



all provide the same **capability**.

Different vehicles

</> Java



```
abstract class Vehicle {  
  
    protected String registrationNumber;  
  
    abstract void start();  
  
    void stop() {  
        // common logic  
    }  
}
```

✓ Abstract class

Because vehicles share common state/behavior.

Order service

</> Java



```
class OrderService {  
  
    private final PaymentProcessor paymentProcessor;
```

```
private final InventoryService inventoryService;  
}
```

✓ Composition

Because:

```
OrderService HAS-A PaymentProcessor  
OrderService HAS-A InventoryService
```



Developer

</> Java

```
class Developer extends Employee {  
}
```



✓ Inheritance

Because:

```
Developer IS-A Employee
```



Best 5-Year Interview Answer

If the interviewer asks:

"When would you choose interface, abstract class, composition or inheritance?"

You can answer:

"I use an interface when I need to define a contract or capability and want implementations to remain loosely coupled. I use an abstract class when closely related classes share common state or implementation and I want to provide partial implementation. I prefer composition when the relationship is HAS-A or when I need flexibility and replaceable components. I use inheritance when there is a genuine IS-A relationship and the subclass can be safely substituted for the parent. In general, I don't use inheritance just for code reuse; I favor composition when the relationship isn't a strong IS-A relationship."

Remember this four-line formula:

Interface → CONTRACT
Abstract class → COMMON BASE
Composition → HAS-A
Inheritance → IS-A



And one senior-level rule:

Need behavior contract? → Interface
Need shared state + logic? → Abstract class
Need to assemble components? → Composition
Need true specialization? → Inheritance



ChatGPT can make mistakes. Check important info.